

Arduino 5 — I²C MPU-6050 Accelerometer & Gyroscope

Tutorial Group: Arduino

Goal

In this tutorial, you will learn how to connect an MPU-6050 motion sensor to the Arduino Mega 2560 using I²C. You will read the sensor's three accelerometer axes, three gyroscope axes, and its internal temperature, then send all seven values to the Arduino Serial Plotter. This tutorial combines I²C communication, sensor data, the non-blocking `millis()` timing technique from the *Arduino 4 — Non-Blocking Timing with `millis()`* tutorial, and Serial Plotter visualization.

What You Need

- Arduino Mega 2560
- USB cable
- Computer with the Arduino IDE
- Breadboard and jumper wires
- MPU-6050 module (GY-521) from the Mega 2560 evaluation kit

Part 1 — What Is the MPU-6050?

The MPU-6050 is a motion sensor containing two sensors in one chip: a 3-axis accelerometer and a 3-axis gyroscope. The accelerometer measures linear acceleration along the X, Y, and Z axes. The gyroscope measures how quickly the sensor is rotating around those same three axes.

That gives us six motion measurements: Accel X, Accel Y, Accel Z, Gyro X, Gyro Y, and Gyro Z, and an internal temperature sensor, giving us a seventh value to observe. The MPU-6050 communicates with the Arduino Mega 2560 through I²C.

The accelerometer and gyroscope together provide six degrees of freedom (6-DOF) of motion measurement.

Part 2 — What Is I²C?

I²C is a communication interface commonly used to connect sensors, displays, and other devices to a microcontroller. Unlike a simple digital output, I²C allows the Arduino Mega 2560 to exchange data with another device.

I²C normally uses two signal lines:

SDA — Serial Data

SCL — Serial Clock

In addition to SDA and SCL, the devices need a common ground and power. The MPU-6050 therefore requires only a small number of connections.

Part 3 — Arduino Mega 2560 I²C Connections

With the Arduino powered off, connect the four required signals shown above. Keep the wires short and make sure the ground connection is solid. The sensor does not require six separate GPIO connections because I²C allows the Arduino to communicate with it using SDA and SCL.

The Arduino Mega 2560 has dedicated I²C pins. On the Mega 2560, SDA is pin 20 and SCL is pin 21.

MPU-6050	Arduino Mega 2560	Purpose
VCC	5.00v	Power
GND	GND	Common ground

SDA	20 (SDA)	I ² C data
SCL	21 (SCL)	I ² C clock

Part 4 — Install the MPU-6050 Library

For this tutorial, use the Adafruit MPU6050 library. It provides a straightforward interface for reading the accelerometer and gyroscope.

- Open the Arduino IDE.
- Open Tools → Manage Libraries (or the Library Manager).
- Search for “Adafruit MPU6050”.
- Install “Adafruit MPU6050” by Adafruit.
- If the IDE asks for required dependencies, install them as well.

Part 5 — Start With I²C and the Sensor

The first part of the program includes the libraries and creates the sensor object:

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

Adafruit_MPU6050 mpu;
```

Wire.h provides the Arduino I²C interface. Adafruit_MPU6050 provides the functions used to communicate with the sensor, and Adafruit_Sensor provides the common sensor data structures used by the library.

Part 6 — Initialize the Sensor

In the code segment below, mpu.begin() attempts to communicate with the MPU-6050. If the sensor cannot be found, the program prints an error and stops. This is useful when troubleshooting wiring or power.

```
void setup()
{
  Serial.begin(115200);

  if (!mpu.begin())
  {
    Serial.println("MPU-6050 not found");
    while (1);
  }

  Serial.println("MPU-6050 found");
}
```

Part 7 — Read the Seven Measurements

The Adafruit library returns accelerometer and gyroscope data as floating-point values. The accelerometer values are reported in meters per second squared (m/s²), while the gyroscope values are reported in radians per second (rad/s), and temperature in Celsius (°C). Note that the temperature sensor is the internal chip die temperature, rather than precise ambient room temperature. If ambient temperature is needed, a different sensor is required.

```
sensors_event_t accel;
sensors_event_t gyro;
sensors_event_t temp;

mpu.getEvent(&accel, &gyro, &temp);
```

This call fills three sensor-event structures. We will use accel for the three accelerometer values, gyro for the three gyroscope values, and temp for the MPU-6050's internal temperature measurement.

Part 8 — Understand the Seven Values

Value	Meaning	Unit
Accel X	Acceleration along X axis	m/s ²
Accel Y	Acceleration along Y axis	m/s ²
Accel Z	Acceleration along Z axis	m/s ²
Gyro X	Rotation rate around X axis	rad/s
Gyro Y	Rotation rate around Y axis	rad/s
Gyro Z	Rotation rate around Z axis	rad/s
Temperature	Internal die sensor temperature	°C

When the board is stationary, the accelerometer measures gravity as part of its acceleration. The total acceleration due to gravity is approximately 9.8 m/s². If one axis is aligned directly with gravity, that axis will show a value near ± 9.8 m/s² while the other two axes will be closer to zero. With other orientations, the gravity measurement may be distributed among all three axes.

The gyroscope values should be near zero when the board is not rotating.

Part 9 — Why We Do Not Use delay()

In earlier Arduino work, delay() may have been used to pause the program. In this tutorial we will deliberately use the non-blocking millis() technique learned in the *Arduino 4 — Non-Blocking Timing with millis()* tutorial. This allows the program to decide when it is time for another sensor reading without stopping the processor with delay().

```
const unsigned long sampleInterval = 50;
unsigned long previousMillis = 0;
```

An interval of 50 milliseconds produces a sampling period of 20 readings per second. The exact rate is not the important lesson here; the important lesson is that the program remains free to do other work between readings.

Part 10 — Create Non-Blocking Sensor Updates

```
void loop()
{
    unsigned long currentMillis = millis();

    if ((currentMillis - previousMillis) >= sampleInterval)
    {
        previousMillis = currentMillis;

        sensors_event_t accel;
        sensors_event_t gyro;
        sensors_event_t temp;

        mpu.getEvent(&accel, &gyro, &temp);

        // Sensor output goes here
```

```
}  
}
```

Notice that there is no `delay()`. `millis()` tells us how much time has passed since the Arduino started. The subtraction `currentMillis - previousMillis` is important because it continues to work correctly when the `millis()` counter eventually rolls over.

Part 11 — Format the Serial Plotter Output

The Arduino Serial Plotter can graph several values at the same time when each line contains labeled numeric fields. We will print seven values on every sample. Keeping the labels consistent makes the seven traces easy to identify.

```
Serial.print("AccelX:");  
Serial.print(accel.acceleration.x);  
Serial.print("\t");  
  
Serial.print("AccelY:");  
Serial.print(accel.acceleration.y);  
Serial.print("\t");  
  
Serial.print("AccelZ:");  
Serial.print(accel.acceleration.z);  
Serial.print("\t");  
  
Serial.print("GyroX:");  
Serial.print(gyro.gyro.x);  
Serial.print("\t");  
  
Serial.print("GyroY:");  
Serial.print(gyro.gyro.y);  
Serial.print("\t");  
  
Serial.print("GyroZ:");  
Serial.print(gyro.gyro.z);  
Serial.print("\t");  
  
Serial.print("Temp:");  
Serial.println(temp.temperature);
```

The `\t` (tab) characters separate the channels. `Serial.println()` ends the sample line. This creates a continuous stream of seven named measurements that the Serial Plotter can graph.

Part 12 — Open the Serial Plotter

1. Upload the completed program to the Arduino Mega 2560.
2. Open Tools → Serial Plotter in the Arduino IDE.
3. Set the baud rate to 115200 if the IDE provides a baud-rate selection.
4. Place the MPU-6050 on a stable surface and observe the traces.
5. Slowly tilt the board in different directions.
6. Rotate the board around each of its three axes.

You should see the accelerometer traces change as the orientation of the board changes. You should see the gyroscope traces respond strongly while the board is being rotated and return toward zero when rotation stops. The temperature trace should change much more slowly than the motion traces.

Part 13 — Experiment With the Sensor

Hold the board still: Observe which accelerometer axis contains most of the gravity measurement and how close the gyro values are to zero.

- Tilt around the X axis: Watch the accelerometer and gyro X/Y/Z traces and determine which channels respond most strongly.
- Tilt around the Y axis: Compare the response with the X-axis experiment.
- Rotate around the Z axis: Watch the gyro Z trace while rotating the board and then stop rotating.
- Move the board without rotating it: Compare the accelerometer response with the gyro response.
- Watch the temperature: Observe how slowly the internal temperature changes compared with acceleration and rotation.
- Change the sample interval: Try 100 ms and 20 ms. Observe how the Serial Plotter trace changes and consider the tradeoff between update rate and processing time.

Part 14 — Complete Program

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

Adafruit_MPU6050 mpu;

const unsigned long sampleInterval = 50;
unsigned long previousMillis = 0;

void setup()
{
  Serial.begin(115200);

  if (!mpu.begin())
  {
    Serial.println("MPU-6050 not found");
    while (1);
  }

  Serial.println("MPU-6050 found");
}

void loop()
{
  unsigned long currentMillis = millis();

  if ((currentMillis - previousMillis) >= sampleInterval)
  {
    previousMillis = currentMillis;

    sensors_event_t accel;
    sensors_event_t gyro;
    sensors_event_t temp;

    mpu.getEvent(&accel, &gyro, &temp);

    Serial.print("AccelX:");
    Serial.print(accel.acceleration.x);
    Serial.print("\t");

    Serial.print("AccelY:");
    Serial.print(accel.acceleration.y);
    Serial.print("\t");

    Serial.print("AccelZ:");
    Serial.print(accel.acceleration.z);
    Serial.print("\t");
  }
}
```

```

    Serial.print("GyroX:");
    Serial.print(gyro.gyro.x);
    Serial.print("\t");

    Serial.print("GyroY:");
    Serial.print(gyro.gyro.y);
    Serial.print("\t");

    Serial.print("GyroZ:");
    Serial.print(gyro.gyro.z);
    Serial.print("\t");

    Serial.print("Temp:");
    Serial.println(temp.temperature);
}
}

```

What You Should Understand

- What I²C is used for.
- The purpose of SDA and SCL.
- Which I²C pins are used by the Arduino Mega 2560.
- What an MPU-6050 contains.
- The difference between an accelerometer and a gyroscope.
- What the X, Y, and Z axes represent.
- Why acceleration and rotation are different measurements.
- Why the accelerometer can measure the effect of gravity.
- Why gyroscope values should approach zero when the sensor is not rotating.
- How an Arduino library provides an interface to a hardware sensor.
- How millis() can be used for non-blocking timing.
- How to send multiple sensor channels to the Serial Plotter.
- What the MPU-6050's internal temperature measurement represents and why it changes more slowly than motion data.
- How to interpret seven simultaneous traces from a motion sensor.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I can connect an MPU-6050 to the Arduino Mega 2560 using I²C.
- ☐ I know that SDA is Mega 2560 pin 20 and SCL is Mega 2560 pin 21.
- ☐ I understand that the MPU-6050 contains a 3-axis accelerometer and a 3-axis gyroscope.
- ☐ I can identify Accel X, Accel Y, Accel Z, Gyro X, Gyro Y, Gyro Z, and Temperature.
- ☐ I understand the difference between acceleration and rotation rate.
- ☐ I understand why gravity appears in the accelerometer readings.
- ☐ I can use the Adafruit MPU6050 library to read the sensor.
- ☐ I can use millis() instead of delay() for periodic sensor sampling.
- ☐ I can send seven labeled values to the Serial Plotter.
- ☐ I understand why the temperature sensor should not be used to measure ambient temperature.
- ☐ I can explain what happens to the seven traces when I tilt or rotate the sensor.

Once this tutorial is complete, you should be comfortable using an I²C motion sensor with the Arduino Mega 2560, reading acceleration, rotation, and the sensor's internal temperature, and producing a continuously updating seven-channel data stream without blocking the program with `delay()`. These skills provide a foundation for using sensor data in later control, monitoring, and competition projects.